LLMs explained (Part 5): Reducing hallucinations by using tools

LLMs still make things up. Can web search and code execution fix that?

JOSE PARREÑO GARCIA
JUN 07, 2025

11 9 Share

This week, we are tackling one of the biggest challenges in LLMs: **getting them to stop making things up.**

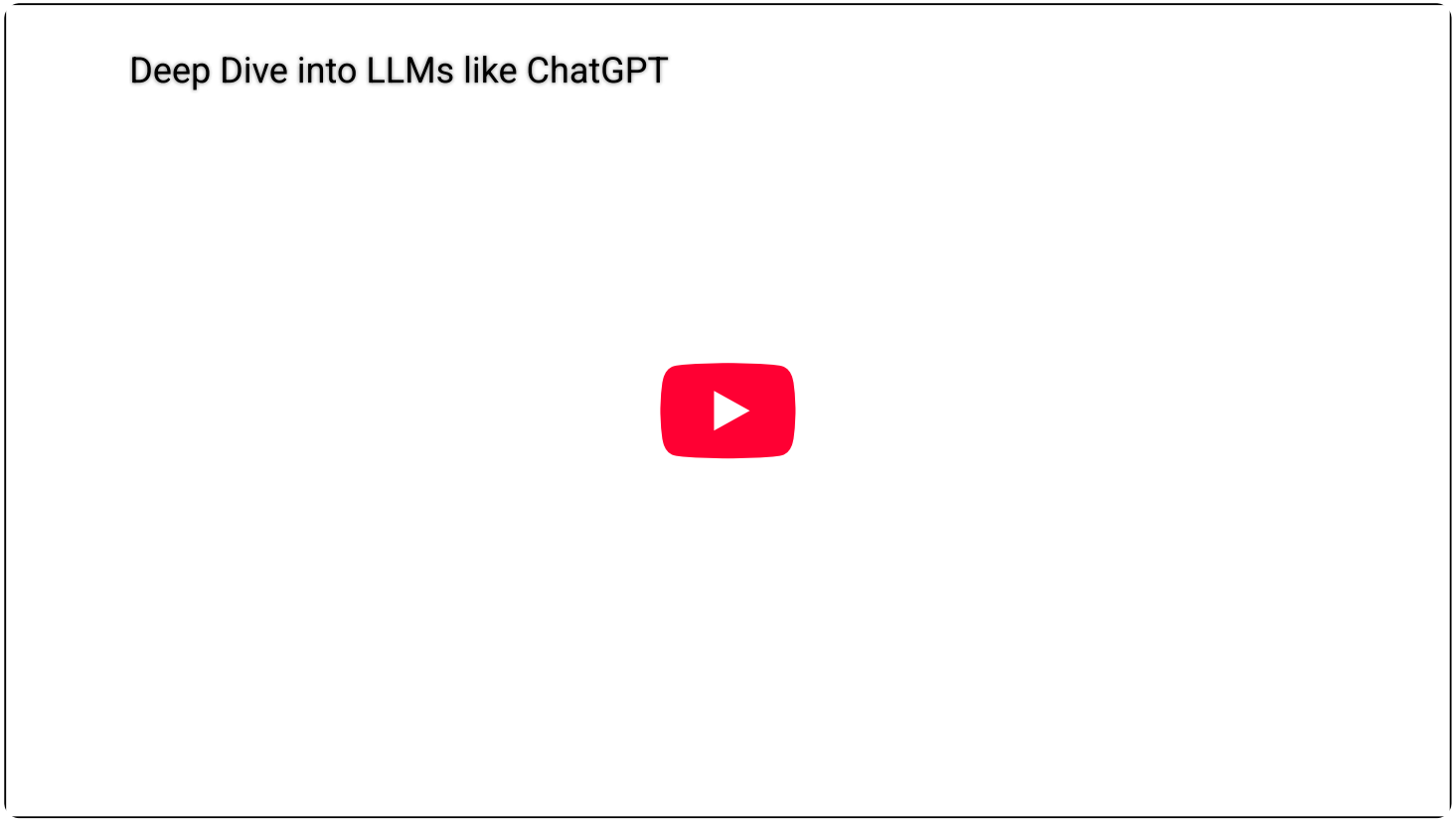
By now, we have covered how LLMs are built, trained, and fine-tuned. But even with all that effort, they still hallucinate. Fine-tuning helps, but, it's not enough. That's why the next step in making LLMs more reliable is to **giving them tools.**

Today, in Part 5, we are diving into how LLMs use **web search and code execution** to reduce hallucinations. These tools help models fetch real-time information and compute precise answers instead of guessing.

Can LLMs finally be trusted to get things right?

Let's find out.

⚡ *Reminder: This series is inspired by Andrej Karpathy's deep dive on LLMs. If you haven't watched it yet, you definitely should!*



Blog series

- ✅ Part 1. LLMs explained: The 3-layer framework behind chatGPT & friends

- ✅ Part 2. LLMs explained: How LLMs collect and clean training data
- ✅ Part 3. LLMs explained: From tokens to training – how a baseli
- ✅ Part 4. LLMs explained: Making LLMs actually useful through fine-tuning
- 📌 **Part 5. LLMs explained: Reducing hallucinations by using tools *(this post!)***
- Part 6. LLMs explained: Smarter AI through Reinforcement Learning *(coming soon!)*

A reminder that fine-tuning helps, but doesn't mitigate all problems.

By now, you know that fine-tuning makes LLMs more useful. In Part 4, we covered how models trained on helpful assistant datasets learn to follow instructions better, reduce mistakes, and become more aligned with what users actually want.

But **fine-tuning isn't a magic fix**. You can't just throw more data at a model and expect it to stop making things up. LLMs still make things up. Why?

- **The internet is always changing.** Training data goes stale as new research, news, and trends emerge.
- **LLMs can't store everything.** Even with billions of tokens, they don't have perfect recall.

- **They generate answers probabilistically.** Instead of retrieving facts, they predict what sounds correct.

To truly reduce hallucinations, we need to go beyond fine-tuning. We need to give LLMs tools to get the right answers *at the moment they're asked*.

In this post we will cover show 2 essential tools that make LLMs more reliable:

1. **Use web search** to fetch real-time information and verify facts
2. **Run a Python interpreter** to compute numbers instead of guessing them.

These tools complement fine-tuned LLMs. Web search prevents outdated knowledge, while code execution ensures mathematical accuracy. Together, they make LLMs more factual, more reliable, and more useful.

What this blog will cover

1. **Introducing (even more) hallucinations.** *LLM's Achilles ankle*
2. **Web search to avoid hallucinations.** *It's a human search on steroids.*
3. **Web search can also be used to stay up to date.** *LLMs have knowledge cut-off dates, so unless they can access real-time info, it is not that useful.*
4. **Models need tokens to think, specially for maths.** *And it's very similar to how human think too!*

5. **Python interpreter to help the model provide maths answers.** *Oh gosh… programming?*

Let's dive in!

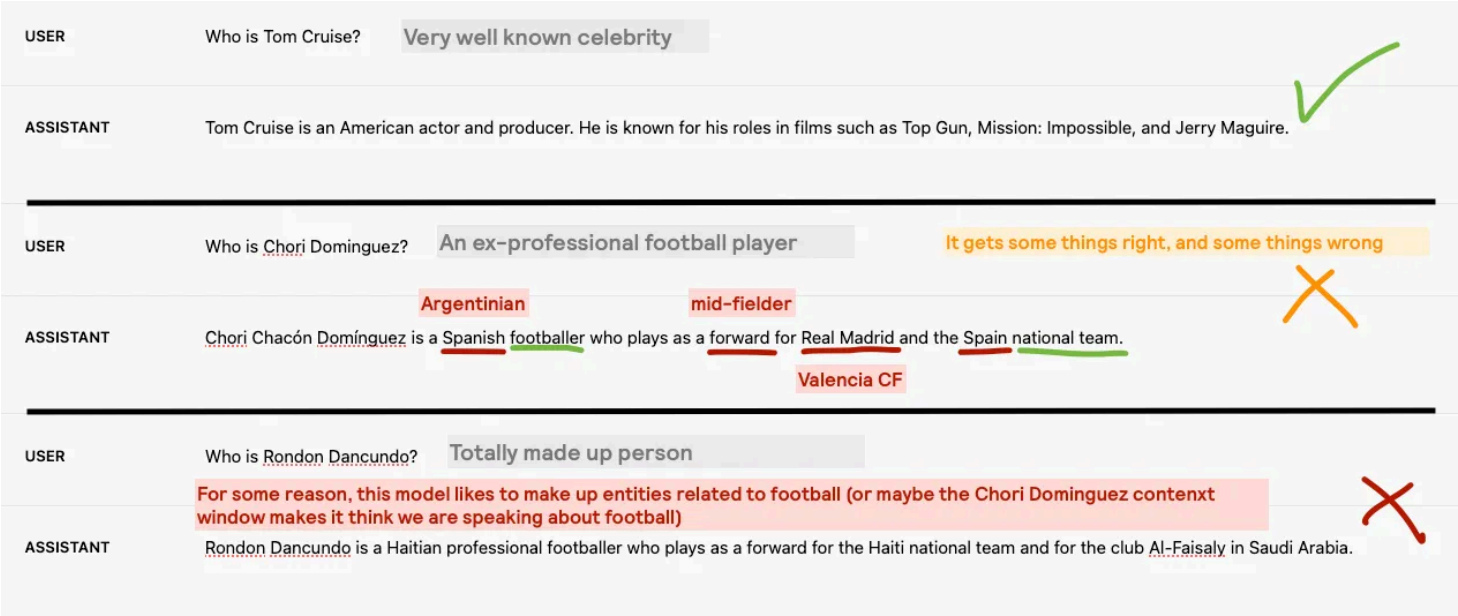
Introducing hallucinations in a bit more depth

LLMs are great at generating human-like text, but they don't actually "know" things. As we have covered in previous blogs, they just predict the most likely next word. This means that when faced with missing knowledge, they might don't stop. They could fill in the blanks, even if what they generate is completely false.

This is what we call a hallucination: when an AI confidently provides an incorrect, misleading, or fabricated answer (even though it sounds really convincing).

What do hallucinations look like?

Let's put an LLM to the test. We will ask it *factual* questions with increasing difficulty and no access to web search (forcing it to rely solely on its training data and internal patterns).



These screenshots are taken using Mistral-7B-Instruct

Why do models get some answers right and some answers wrong?

Knowing that LLMs are built on statistical patterns, then it should be no surprise that, if the model has a lot of sequence tokens in its internal knowledge (ie, Tom Cruise) it can retrieve a solid factual answer. The less times a token sequence happens, the less accurate the answer will be. We can reach a point where the sequence token (ie, Rondon Dancundo) has never happened, but still the LLM provides a best guess answer.

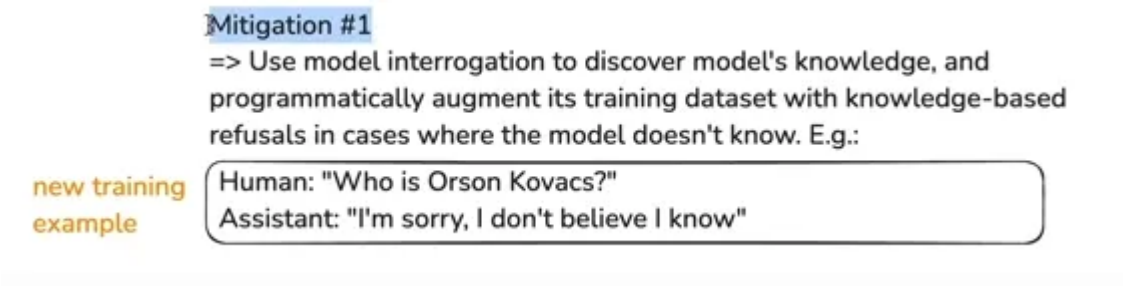
Scenario	Accuracy	Why This Happens?
✅ **Famous Person (Easy)**	✅ **Accurate**	- Tom Cruise is **widely known**, appearing **thousands of times** in the training data. - The model has **high-confidence token probabilities**, so there's little ambiguity.
🤔 **Obscure Person (Uncertain)**	✅ **Mostly correct** – might get **minor details wrong**. ❌ **Or might mix up** facts with another person.	- "Chori Domínguez" is **less famous**, meaning references in the training data are **scarce**. - The model **may misattribute** facts (e.g., confusing players with similar names). - The model has **lower-confidence token probabilities**, relying on **loose associations** rather than strong facts.
🚨 **Completely Fake Person (Total Hallucination)**	❌ **Completely fabricated.**	- The model has **never seen this name before**, but instead of saying *"I don't know"*, it **creates a false answer**. - It follows **patterns from similar-sounding names** (e.g., "Borromeo" and "Dancundo" resemble real historical figures). - LLMs **always generate something**—they don't naturally admit uncertainty.

The "I don't know" problem

Humans are comfortable saying *"I don't know"* when they lack information. LLMs, on the other hand, are not designed to stop generating text, so, they fill in the gaps rather

than admitting uncertainty.

One solution is **teaching LLMs to detect when they are unsure** and respond accordingly. This is what we covered in Part 4, where human labellers created lots of conversations that the LLM can learn from. As part of this new corpus of data, there will be lot's of examples where "I dont know" is the correct answer. Below, you can see a screenshot from Karpathy's video.



However, this mitigation strategy does not scale for 2 reasons:

1. You can't cover all possible knowledge gaps in the world through fine-tuning "I don't know answers".
2. Even if you could, you would need to fine-tune again every time new gaps are discovered.

So, if fine-tuning isn't enough, what is the fix? Instead of training a model to remember everything, what if we just let it look things up? Just like humans do? This

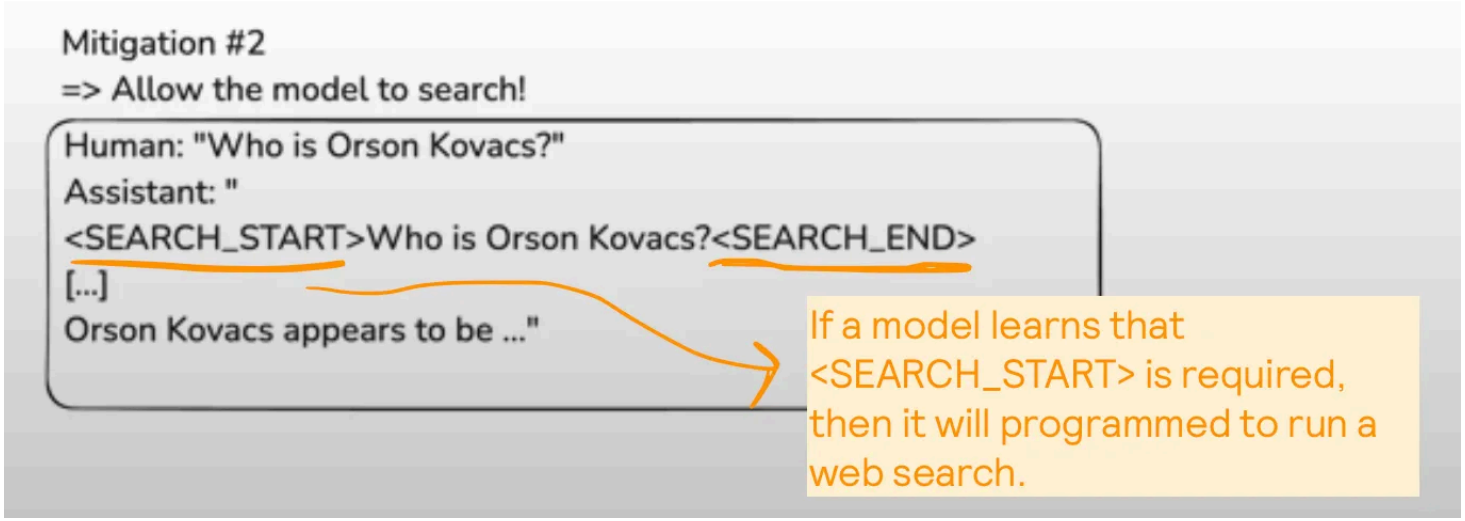
is where **web search changes the game.**

Web search to avoid hallucinations.

When you and I don't know the answer to something, what do we do? We Google it. We open multiple tabs, skim through several articles, and try to figure out which sources are credible.

Well, we can train an LLM to do this, but on *steroids*.

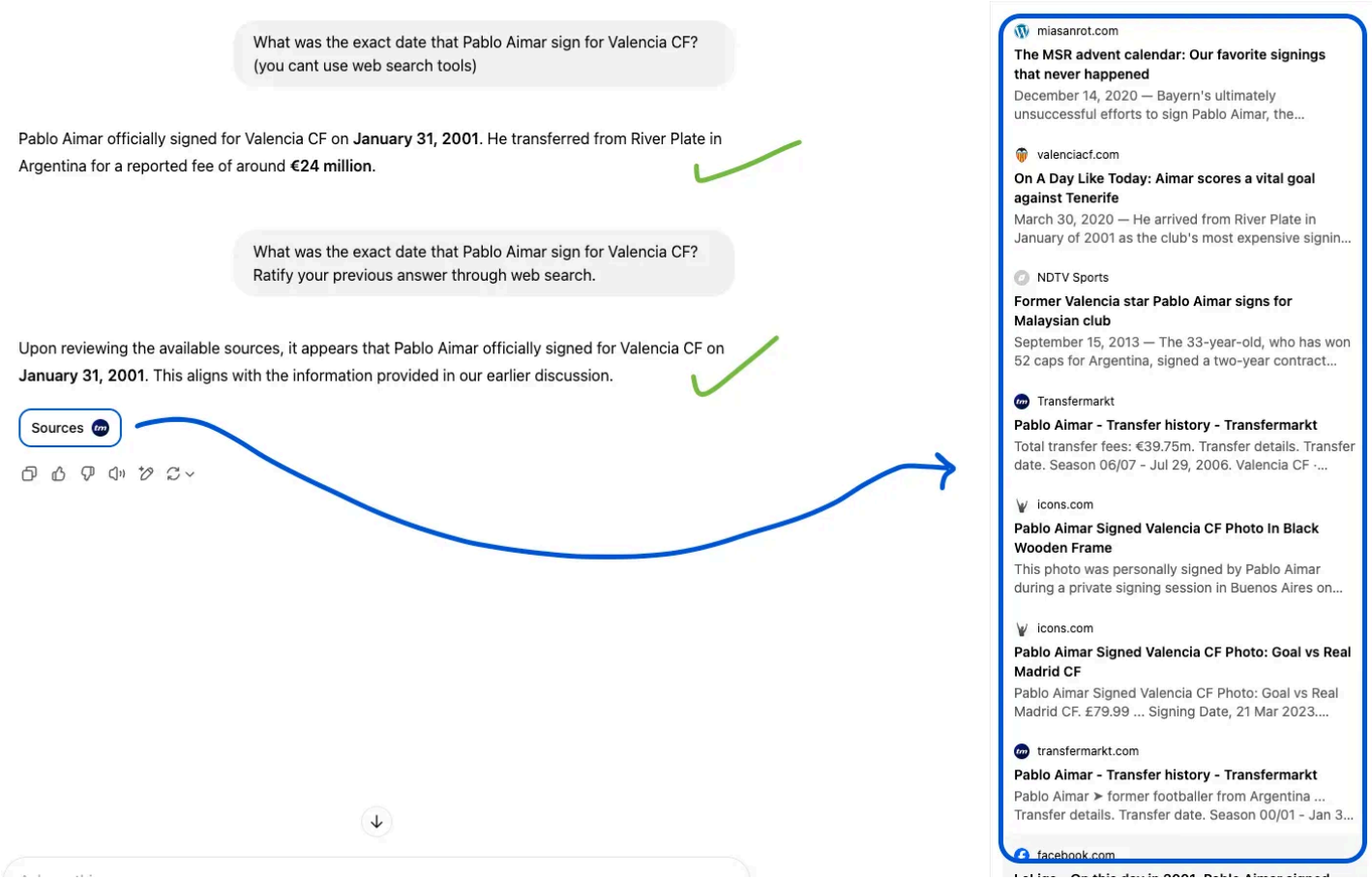
Remember that in part 4 we introduced the Conversation Protocol where special tokens (`<|im_start|>` & `<|im_end|>`) indicated a human → assistant conversation. We use the same technique to train an LLM to activate a search tool. The screenshot below (from Karpathy's video) shows this in action.



What really happens when the LLM triggers a web search?

Alright, so we've trained an LLM to search like a human. But what's actually happening behind the scenes?

Let me show you first through an example. The screenshot below answered a question without the need of search (I couldn't come up with a question that ChatGPT didn't know the answer to!), so I also asked it to ratify the answer by running an online search.

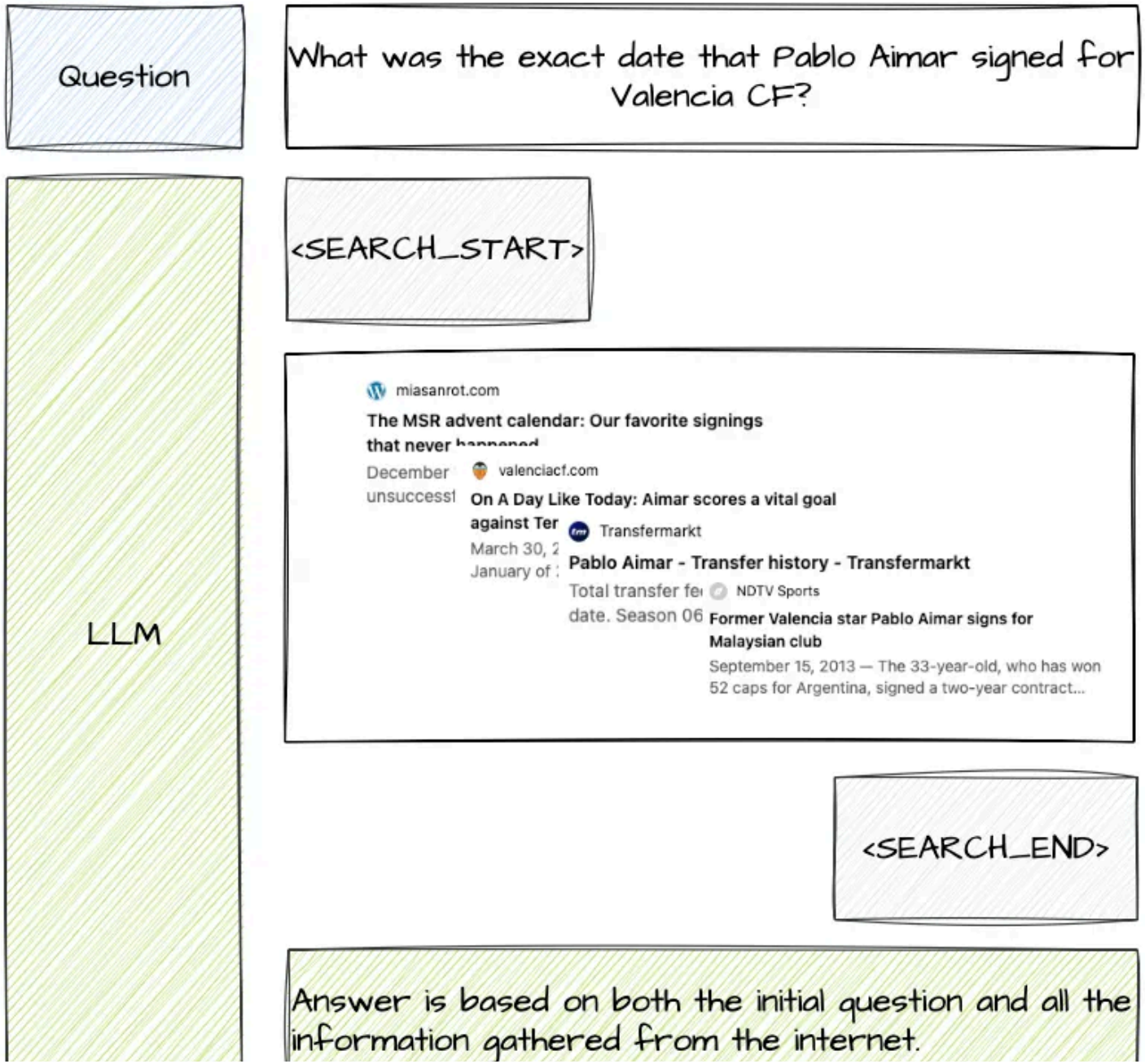


To the right of the chat, there is a list of sources that ChatGPT went through. There are total of 9. Now, what is ChatGPT doing with these sources?

It’s simpler than you might think:

1. **Retrieves key passages:** The LLM likely extracts important snippets from search results.
2. **Basic cleaning:** Some preprocessing happens (similar to the data cleaning from Part 2, but simpler).
3. **Tokenization:** The extracted text gets converted into tokens, allowing the LLM to process it as a long sequence tokens.
4. **Synthesizing a final answer:** The LLM **distills all the retrieved data** and generates a response.

In other words, the LLM is using your input question, expanding its context with additional retrieved text, and then generating a more informed response. The screenshot below shows the schematic diagram of how the LLM would use web search to answer questions.


Question
What was the exact date that Pablo Aimar signed for Valencia CF?
LLM
<SEARCH_START>
miasanrot.com
The MSR advent calendar: Our favorite signings
that never
December
unsuccessf
valenciacf.com
On A Day Like Today: Aimar scores a vital goal
against Ter
March 30, 2
January of
Transfermarkt
Pablo Aimar - Transfer history - Transfermarkt
Total transfer fe
date. Season 06
NDTV Sports
Former Valencia star Pablo Aimar signs for Malaysian club
September 15, 2013 — The 33-year-old, who has won 52 caps for Argentina, signed a two-year contract...
<SEARCH_END>
Answer is based on both the initial question and all the information gathered from the internet.

Basically, the LLM is doing what you or I would do—but much, much faster. Imagine opening 9 web pages at once, skimming them in two seconds, and summarising the answer instantly. That's what's happening under the hood.

However, web search isn't *just* about fact-checking **static** knowledge—it also solves another major issue: fact-checking **dynamic** knowledge. For example, what happens if we ask a factual question regarding an event that happened after its training data was cut off?

That is exactly what we will cover in the next section.

Web search can also be used to stay up to date.

If you ask an LLM about a recent event, it has two options: make something up or admit, "I don't know". This happens because an LLM's "knowledge" isn't updated in real time; it's frozen at a specific point when the model was trained. This point is called the **cutoff date**—the last moment it was exposed to new information..

The screenshot below shows how ChatGPT overcomes this limitation—by searching the web for information beyond its cutoff date.

What is you knowledge cutoff date?

My knowledge cutoff date is **June 2024**, but I can fetch real-time information from the web if you need the latest updates. Let me know what you are looking for!

Event happened after cutoff date

When did the 2025 super bowl final happen? And who won the final? If you need to use a web search tool, please specify it clearly at the beginning of your response.

The 2025 Super Bowl, officially known as Super Bowl LIX, took place on **February 9, 2025**, at the Caesars Superdome in New Orleans, Louisiana. In this matchup, the Philadelphia Eagles defeated the Kansas City Chiefs with a final score of **40–22**, securing their second Super Bowl title in franchise history. en.wikipedia.org +10

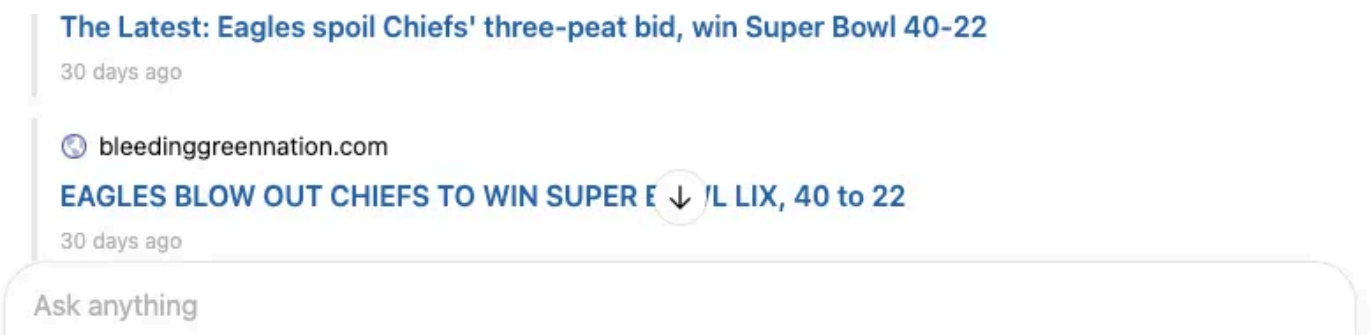
The Eagles dominated the game from the outset, leading 24–0 by halftime. Quarterback Jalen Hurts delivered an outstanding performance, throwing for 221 yards and two touchdowns, and rushing for an additional score, earning him the Super Bowl MVP honors. people.com +1 apnews.com

This victory prevented the Chiefs from achieving a three-peat, as they were aiming for their third consecutive Super Bowl win. The Eagles' defense played a crucial role, effectively limiting the Chiefs' offense and contributing to the decisive outcome. apnews.com +2

Eagles Triumph in Super Bowl LIX Over Chiefs

So ChatGPT decided to run a web search

AP apnews.com



As you have been reading, tooling like web search, can help LLMs become more factual in their responses. By recognising when to say “I don’t know” and when to search the web, LLMs become far more reliable assistants.

But factual accuracy isn’t the only challenge. LLMs also struggle with something much simpler: basic math. In Part 4, we touched on this issue when discussing how fine-tuning improves reasoning, but even the best models still make simple arithmetic mistakes. Let’s explore why.

Models need tokens to think, especially for maths.

If LLMs are trained on billions of examples, why do they still struggle with basic math? Let’s re-use the screenshot from part 4. When I asked the LLM to solve a math problem in one step, it got the wrong answer twice. But when I let the LLM explain its reasoning step by step, it got the answer right.

USER

Emily buys 23 apples and 177 oranges. Each orange costs 4 dollars. The total cost of all fruit is 869 dollars. What is the cost of each apple?

Answer the question with 1 single token. Give me the answer and nothing else.

ASSISTANT

5

USER

Emily buys 23 apples and 177 oranges. Each orange costs 4 dollars. The total cost of all fruit is 869 dollars. What is the cost of each apple?

Answer the question with 1 single token. Give me the answer and nothing else.

ASSISTANT

29

USER

Emily buys 23 apples and 177 oranges. Each orange costs 4 dollars. The total cost of all fruit is 869 dollars. What is the cost of each apple?

Now, you can use as many tokens as you need to think the answer through.

ASSISTANT

To find the cost of each apple, we need to first calculate the total cost of the oranges and then subtract that from the total cost of all fruit.

The total cost of the oranges is the number of oranges multiplied by the cost of each orange:
177 oranges * 4 dollars/orange = 708 dollars

Now, we subtract the total cost of the oranges from the total cost of all fruit to find the total cost of the apples:
869 dollars - 708 dollars = 161 dollars

This means that the total cost of the 23 apples is 161 dollars. To find the cost of each apple, we divide the total cost of the apples by the number of apples:
161 dollars / 23 apples = 7 dollars per apple

So, the cost of each apple is 7 dollars.

Why does this happen?

First, think of an LLM like a person solving a math problem in their head. If you ask someone "What's 234 × 89?" and demand an instant answer, they might guess or make

a mental mistake. But if you let them write down the calculation step by step, they are much more likely to get it right.

The same applies to LLMs.

1-pass vs. Multi-step thinking

🔷 **When you ask for a single-token answer**

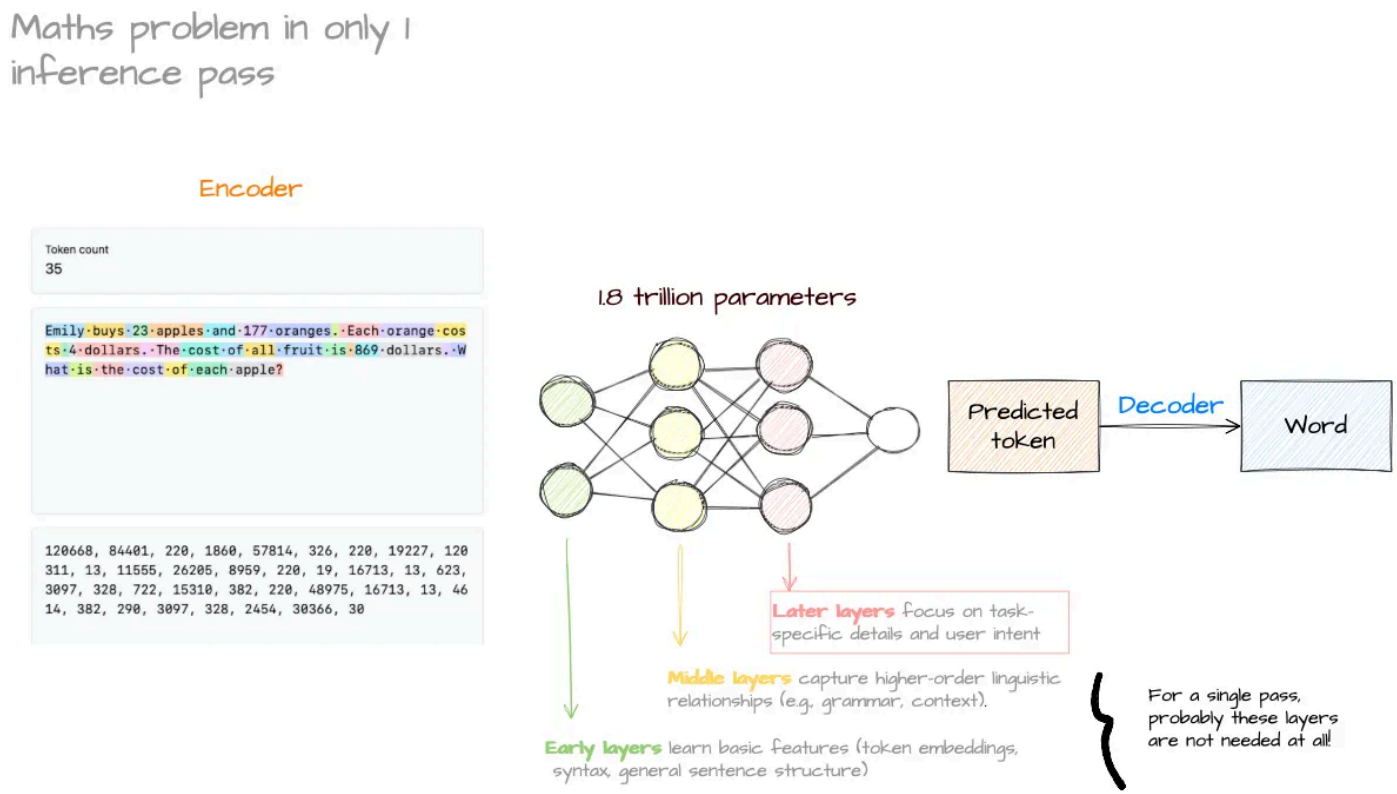
- The LLM has to crunch the entire solution in one prediction step.
- If it miscalculates or misrepresents a number, the answer is wrong—and there is no chance to correct it.

The diagram below shows the following:

- The LLM gets as an input 35 tokens.
- We are asking the LLM to get the answer right in the *next* token
- The LLM can only do a single pass through the Neural Network
- We are basically asking the LLM to ditch any previous layers and rely solely on the later layers (the ones which control specific tasks / knowledge such as maths).

Really, if we wanted the LLM to get this right in 1 go, the LLM will have had to see this specific 35 token sequence in its training dataset. Not something similar. The

exact token sequence. And that is highly improbable.



Single pass prediction diagram

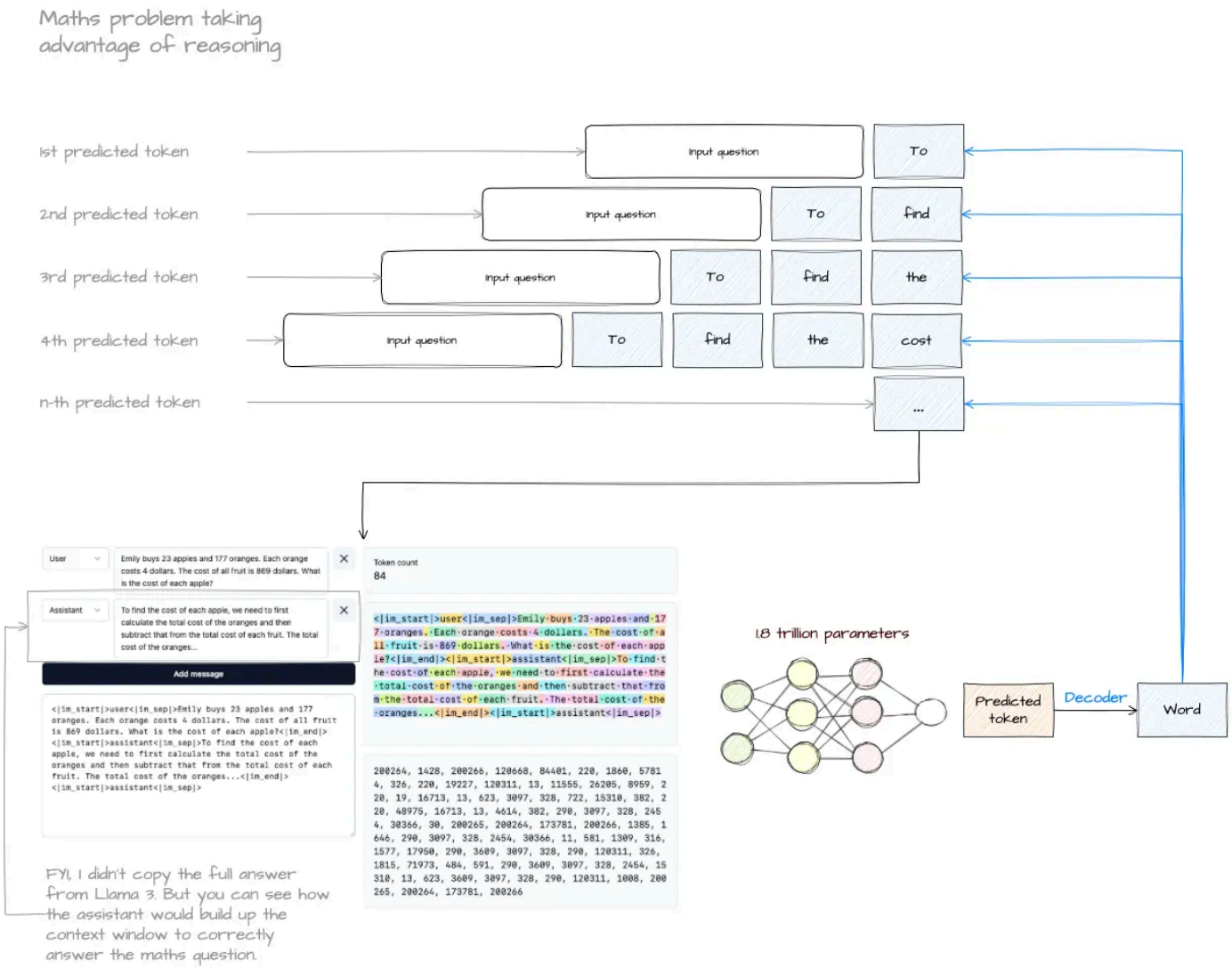
🔷 When you let the LLM generate multiple tokens

- It can break the problem into smaller steps.

- With more tokens, there is higher chance of generating a sequence that is similar to other maths problems it has seen in memory.

The diagram below shows the following:

- The LLM takes advantage of all the neural network layers to start building a longer context window.
- The LLM is probably building this context window mimicking how fine-tuned examples for maths have been crafted.
- By building a longer context window, the LLM will have the chance to find similar maths problems in its memory.



Multiple passes prediction diagram

But, LLMs still aren't calculators!

Even with multi-token generation, they don't calculate like a real math engine. Remember, they generate text based on probability, so they might still make mistakes.

So, is there a way where we can have a better chance at solving maths problems? Yes, through code. Let's look at this in the next section.

Python interpreter to help the model provide maths answers.

As we have seen, a mitigation strategy for LLMs to provide good maths (or science) answers, is to let is "think". By thinking we mean having longer context windows to support its answer. However, it can still get it wrong, as the answer is still based on statistical patterns.

However, there is a better option: building and running code.

ChatGPT has in-built the possibility of running Python scripts. LLMs are really well at translation between languages; and English to Python is still a language translation problem. So, we can be really confident that smaller pieces of code are well executed.

Below, you can see a screenshot where ChatGPT generates a python script to answer the question.

Emily buys 23 apples and 177 oranges. Each orange costs 4 dollars. The total cost of all fruit is 869 dollars. What is the cost of each apple?

Use code to aid in your response.

Analysis

python

Always show details Copy

```python
# Given data
num_apples = 23
num_oranges = 177
cost_per_orange = 4
total_cost = 869

# Calculate the total cost of oranges
total_cost_oranges = num_oranges * cost_per_orange

# Calculate the cost of apples
total_cost_apples = total_cost - total_cost_oranges

# Cost per apple
cost_per_apple = total_cost_apples / num_apples
cost_per_apple
```

Result

7.0

Key takeaways from part 5

1. **LLMs are still prone to hallucinations.** Even fine-tuned models can generate incorrect or misleading information because they predict the next word probabilistically rather than retrieving facts.
2. **Fine-tuning alone isn't enough to prevent errors.**
   Tools like search engines and code execution complement fine-tuning, giving LLMs external ways to verify and compute rather than relying purely on their internal training data.
3. **Web search reduces outdated or incorrect responses.** Just like humans look things up, LLMs can improve factual accuracy by fetching real-time information instead of relying solely on pre-trained knowledge.
4. **Math is still a weak spot—unless LLMs use code.** LLMs struggle with precise calculations because they rely on pattern prediction, not arithmetic reasoning. Running a Python interpreter allows models to generate mathematically correct answers instead of guessing.

Now, I want to hear from you!

📢 LLMs are powerful, but they still struggle with factual accuracy.

- What do you think about LLMs using external tools like web search and code execution?
- Do you trust AI more when it can fetch real-time information?
- Or do you think hallucinations will always be an issue, no matter the tools?

Drop your thoughts in the comments! Let's make this a conversation. 👇

See you in the next post! 👋

Further reading

If you are interested in more content, here is an article capturing all my written blogs!

All my articles, one convenient place

JOSE PARREÑO GARCIA • 15 OCTOBER 2024

Read full story →

Senior Data Science Lead is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

11 Likes · 9 Restacks

Discussion about this post

Comments Restacks

Write a comment...

© 2026 Jose Parreño Garcia · Privacy · Terms · Collection notice
Substack is the home for great culture